Tarea 1: Caracterización espectral y prototipado de filtros para reducción de ruido

Instituto Tecnológico de Costa Rica – Escuela de Ingeniería en Computadores
Análisis de Señales Mixtas

Estudiante: Isaac Somarribas Montero
Profesor: Luis Chavarría Zamora

Resumen

En este trabajo se analizan tres señales con ruido y se comparan dos métodos para reducirlo: un filtro FIR pasa bajas de 53 coeficientes y un método espectral por bloques con FFT/IFFT. Se evalúan la SNR, el error, la conservación de la señal útil, el costo de cálculo y la memoria. El FIR funciona mejor cuando la interferencia está fuera de la banda útil; el método espectral ofrece mejores resultados con ruido blanco y de banda limitada, aunque necesita más memoria y tiene mayor latencia. Finalmente, se propone una implementación de referencia con tres nodos y frecuencia de muestreo de 8 kHz.

Alcance, convenciones y supuestos

- Se usa $f_s = 8$ kHz, una señal útil de tres tonos (500, 1200 y 2000 Hz), $L = 16384$ muestras (2.048 s) y bloques de $N = 512$ con salto de 256 muestras. Como la frecuencia más alta es 3100 Hz, no se supera el límite de Nyquist de 4 kHz. En hardware se requiere un filtro antialiasing antes del ADC.
- Las métricas se calculan en el segmento interior, descartando 512 muestras en cada extremo. En el FIR se compensa el retardo de grupo de 26 muestras.
- Las señales se generaron con el notebook del profesor y se guardaron con una SNR objetivo de 5 dB. Se usaron tres semillas y los casos de ruido tonal, blanco y de banda limitada.
- Como el enunciado no especifica hardware, se toma como referencia un STM32F407 con Cortex-M4F a 168 MHz y 192 kB de SRAM [**?**]. Los tiempos del microcontrolador son estimaciones y deben medirse en el hardware real.
- Las figuras y tablas se generan desde el notebook. El código y las 36 pruebas automáticas se encuentran en `src/` y `tests/`.

1 Fundamentos, muestreo y FFT (2.1)

1.1 Serie de Fourier, transformada, DFT y FFT

Una señal que se repite cada cierto tiempo T_0 se puede representar con la **serie de Fourier** $x(t) = \sum_m c_m e^{j2\pi mt/T_0}$, con $c_m = \frac{1}{T_0}\int_{T_0} x(t)e^{-j2\pi mt/T_0}dt$: un espectro discreto en múltiplos de $1/T_0$.

La **transformada de Fourier** $X(f) = \int x(t)e^{-j2\pi ft}dt$ usa la misma idea para señales que no se repiten (espectro continuo). Al muestrear con f_s el espectro se repite con periodo f_s y se obtiene la **DTFT** $X(e^{j\omega}) = \sum_n x[n]e^{-j\omega n}$. La **DFT** de N muestras,

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \qquad k = 0, \ldots, N-1, \tag{1}$$

calcula la DTFT en N frecuencias separadas por la misma distancia $\omega_k = 2\pi k/N$, y equivale a la serie de Fourier de la extensión periódica de esas N muestras: $X[k]/N = c_k$ si la ventana contiene un número entero de periodos.

Las cuatro primeras son *transformaciones* (definen qué espectro se obtiene). La **FFT** no es otra transformada: es un *algoritmo* que calcula exactamente la DFT factorizando la matriz de Fourier (Cooley–Tukey [2]); con $N = 2^m$ reduce el costo de N^2 a $\frac{N}{2}\log_2 N$ multiplicaciones complejas (para $N = 512$: 262 144 frente a 2 304, un factor 114). El resultado es prácticamente el mismo; la pequeña diferencia se debe al redondeo.

Ejemplo numérico. $x(t) = 1 + 2\cos(2\pi\,1000t) + 0.5\sin(2\pi\,2000t)$, $f_s = 8$ kHz, $N = 64$ (8 periodos exactos). Sus coeficientes de Fourier son $c_0 = c_{\pm1} = 1$, $c_2 = -0.25j$ y $c_{-2} = +0.25j$; el bin de la armónica m es $k = m\,f_0/\Delta f = 8m$ (módulo 64).

m	bin k	c_m (serie)	$X[k]/N$ (DFT)	|error|
0	0	1.000	1.000	6.8×10^{-16}
1	8	1.000	1.000	2.1×10^{-15}
-1	56	1.000	1.000	2.2×10^{-15}
2	16	$-0.000 - 0.250j$	$-0.000 - 0.250j$	6.3×10^{-16}
-2	48	$0.000 + 0.250j$	$-0.000 + 0.250j$	6.4×10^{-16}

La DFT calculada con la FFT propia reproduce los coeficientes con un error máximo del orden de 10^{-15}, lo que confirma $X[k]/N = c_m$. Para comparar los algoritmos se implementaron la DFT por definición y una FFT radix-2 propia, y se contrastaron con `numpy` [**?**]. Para $N = 512$, la FFT propia coincide con `numpy` con un error máximo de 3.6×10^{-14} y con la DFT directa con 1.2×10^{-11}. La DFT directa requiere más operaciones (N^2 frente a $\frac{N}{2}\log_2 N$), por lo que la FFT resulta más conveniente para bloques grandes. Los tiempos medidos en Python sirven solo como referencia.

1.2 Resolución, latencia y memoria

Una ventana de duración $T = N/f_s$ solo puede distinguir frecuencias separadas aproximadamente $1/T$; por eso las muestras espectrales quedan separadas

$$\Delta f = \frac{f_s}{N} = \frac{1}{T}. \tag{2}$$

Con $f_s = 8$ kHz y $N = 512$ resulta $\Delta f = 15.62$ Hz y $T = 64$ ms.

N	Δf [Hz]	$T = N/f_s$ [ms]	Salto [ms]	RAM aprox. [kB]
256	31.25	32	16	4
512	15.62	64	32	8
1024	7.81	128	64	16
2048	3.91	256	128	32

Hay que tomar en cuenta lo siguiente: para distinguir mejor las frecuencias se necesita un bloque más largo, y eso (i) aumenta la *latencia*, porque no hay salida hasta tener las N muestras (T,

más el procesamiento); (ii) aumenta la *memoria* linealmente (con 4 arreglos de N valores float32 son $\approx 16N$ B: 8 kB para $N = 512$ y 64 kB para $N = 4096$, un tercio de la SRAM supuesta) y el cómputo como $N \log_2 N$; y (iii) promedia señales no estacionarias, de modo que los transitorios se desdibujan. El relleno con ceros interpola la forma del espectro pero no mejora la resolución real.

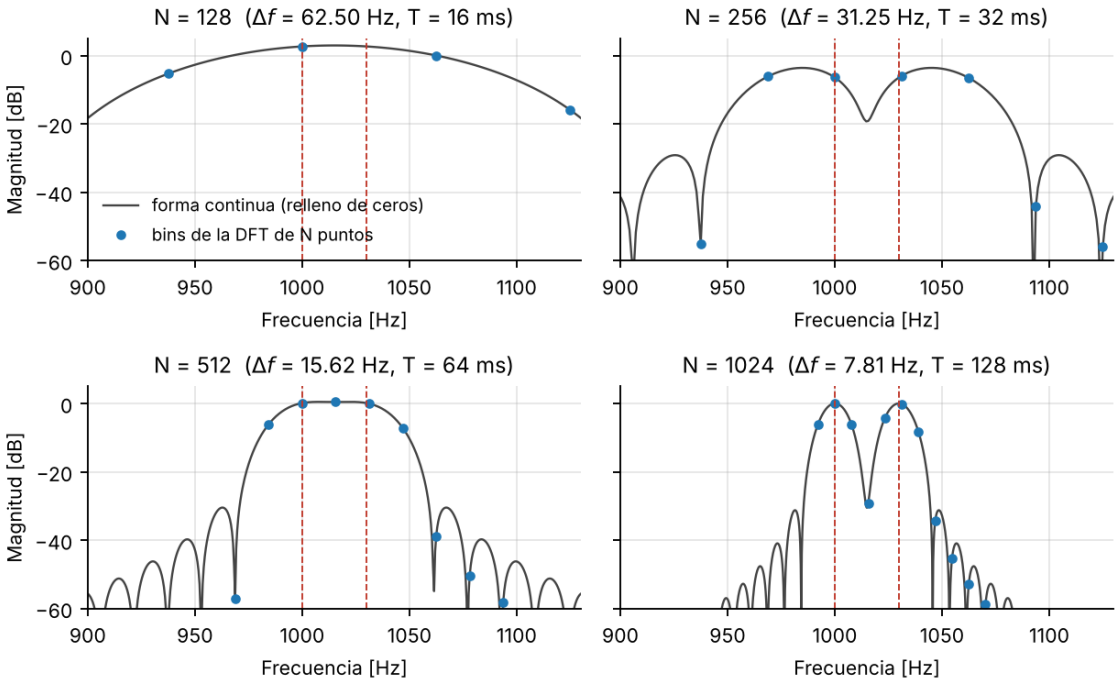


Figura 1: Dos tonos (1000 y 1030 Hz) con ventana Hann para distintos N. Los puntos son los bins de la DFT de N puntos; la línea es la forma continua (relleno de ceros).

En la Fig. 1, los tonos están separados 30 Hz. Con $N = 1024$ se distinguen mejor que con bloques pequeños, aunque aumenta la latencia y la memoria. El relleno con ceros hace que la curva se vea más suave, pero no mejora la resolución real.

1.3 Fuga espectral: tono alineado frente a tono entre bins

Se generaron un tono en $k = 64$ (1000 Hz, alineado con un bin) y otro en $k = 64.5$ (1007.8125 Hz, entre dos bins), ambos con $N = 512$ y fase inicial $\pi/4$. La magnitud se normaliza por $A \sum w[n]/2$, así 0 dB es la amplitud verdadera.

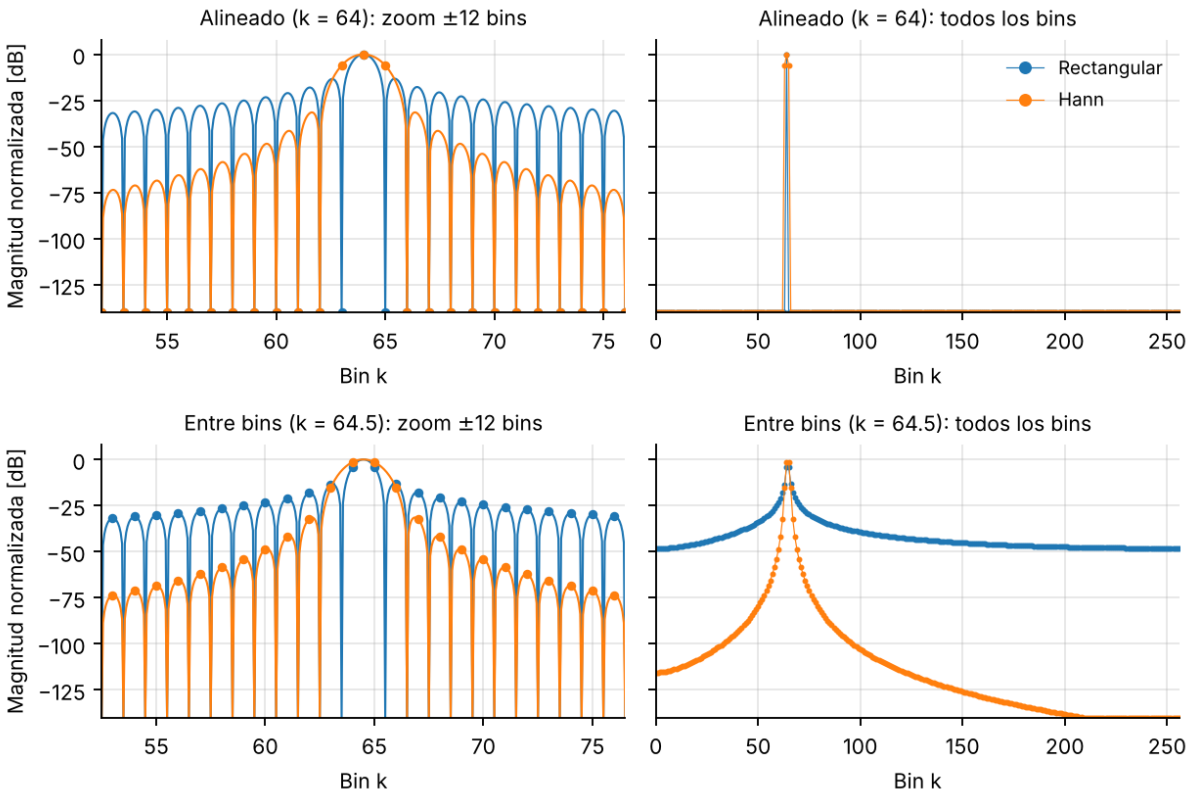


Figura 2: Magnitud normalizada con ventana rectangular y Hann. Arriba: tono alineado. Abajo: tono entre bins. Izquierda: forma continua y bins; derecha: bins de toda la banda.

Ventana	Ancho -3 dB [bins (Hz)]	Nulo a nulo [bins (Hz)]	Lóbulo lateral máx. [dB]	Caída [dB/oct]
Rectangular	0.89 (13.8)	2.0 (31)	-13.3	6
Hann	1.44 (22.5)	4.0 (62)	-31.5	18

Ventana	Pérdida por picoteo, tono entre bins [dB]	Energía fuera de ± 2 bins, alineado [%]	Energía fuera de ± 2 bins, entre bins [%]
Rectangular	-3.90	0.000	8.267
Hann	-1.42	0.000	0.051

Interpretación. La ventana rectangular tiene un lóbulo principal más estrecho, pero sus lóbulos laterales son mayores y producen más fuga espectral cuando el tono cae entre bins. La ventana Hann reduce esos lóbulos laterales y la fuga, aunque ensancha el lóbulo principal. Por eso se eligió Hann para el método espectral; con un salto de $N/2$ también permite reconstruir la señal mediante solapamiento y suma (Sección 4).

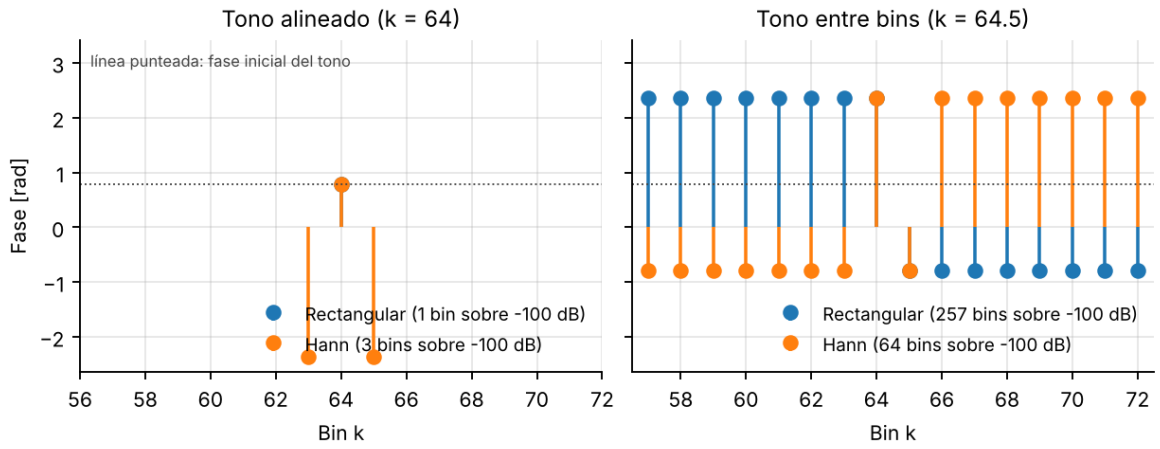


Figura 3: Fase de los bins con magnitud mayor que -100 dB. La línea punteada es la fase inicial del tono ($\pi/4$).

Fase. En un tono alineado, la fase del bin central coincide con la fase inicial. Cuando el tono cae entre bins, la fase cambia entre los bins vecinos. Por eso la fase se interpreta principalmente cerca de los picos y no resulta útil para separar ruido bin a bin.

1.4 Simetría conjugada

Para $x[n] \in \mathbb{R}$: $X[N-k] = X^*[k]$; la magnitud es par y la fase impar respecto de $N/2$ (Fig. 4). Numéricamente, $\max |X[N-k] - X^*[k]| = 8.9e-15$ (tonos y señal aleatoria, $N = 64$).

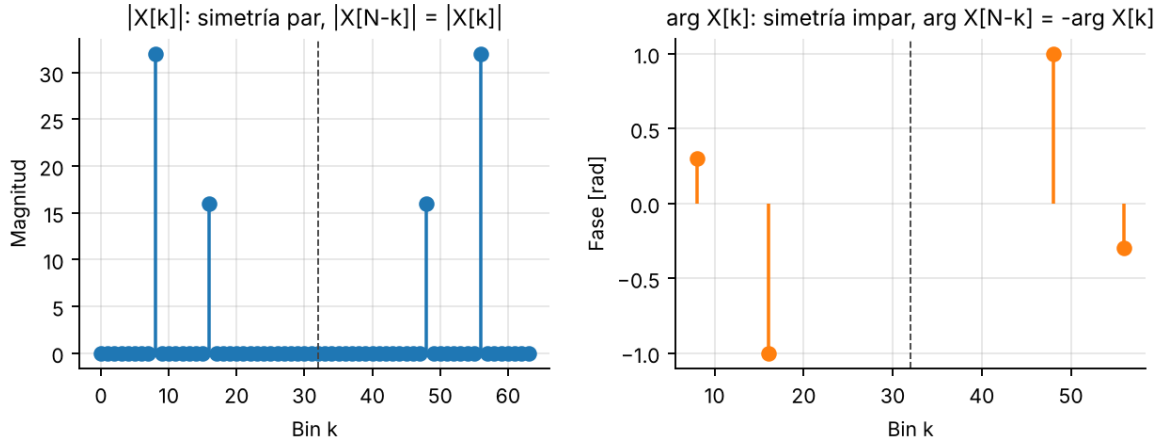


Figura 4: Espectro completo ($N = 64$) de dos tonos: simetría par de la magnitud e impar de la fase.

Importancia de la simetría. Para que la IFFT produzca una señal real, los bins conjugados deben modificarse por pares. Si se elimina un bin sin eliminar su espejo, aparece una parte imaginaria y el tono no se cancela correctamente. Por eso se usa $G[N-k] = G[k]$.

2 Generación y caracterización de señales (2.2)

Se prepararon tres casos reproducibles con semillas distintas. Cada uno guarda la señal limpia, el ruido, la señal contaminada y sus metadatos. El nivel de ruido se ajustó para obtener una SNR objetivo de 5 dB.

Caso	Tipo de ruido	SNR objetivo [dB]	Semilla
1	Interferencia senoidal de 3100 Hz	5.0	101
2	Ruido blanco gaussiano	5.0	202
3	Ruido de banda limitada, 1800–2800 Hz	5.0	303

Verificación de la SNR. En doble precisión se obtiene el valor objetivo de 5 dB. Al guardar las señales como WAV de 16 bits, aparecen diferencias pequeñas por la cuantización, menores que 0.001 dB.

Señal temporal, magnitud, fase y distribución de energía.

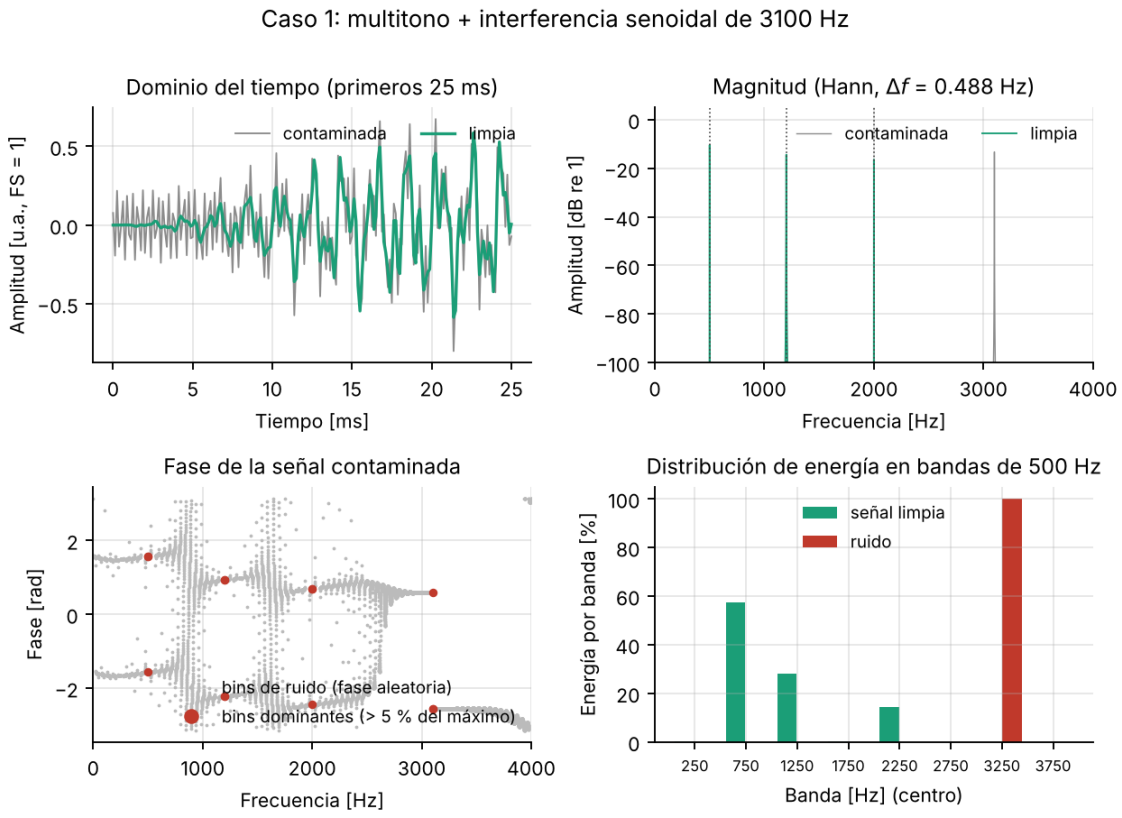


Figura 5: Caso 1: multitono + interferencia senoidal de 3100 Hz.

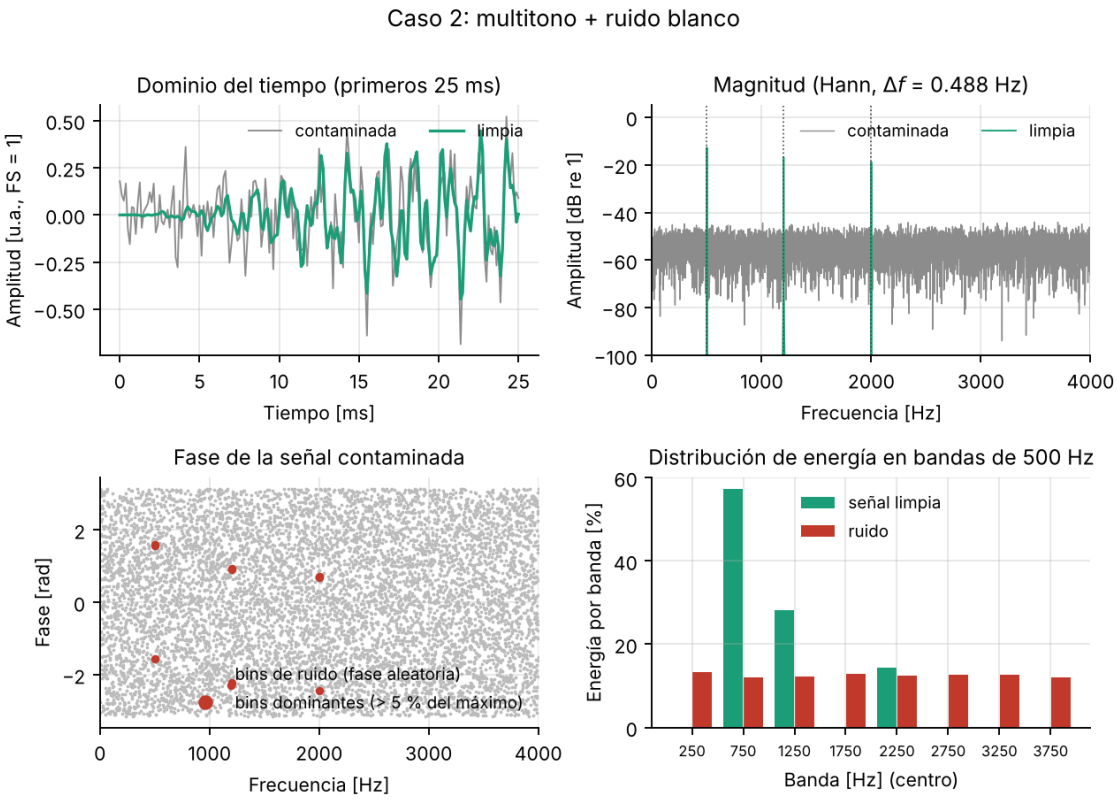


Figura 6: Caso 2: multitono + ruido blanco.

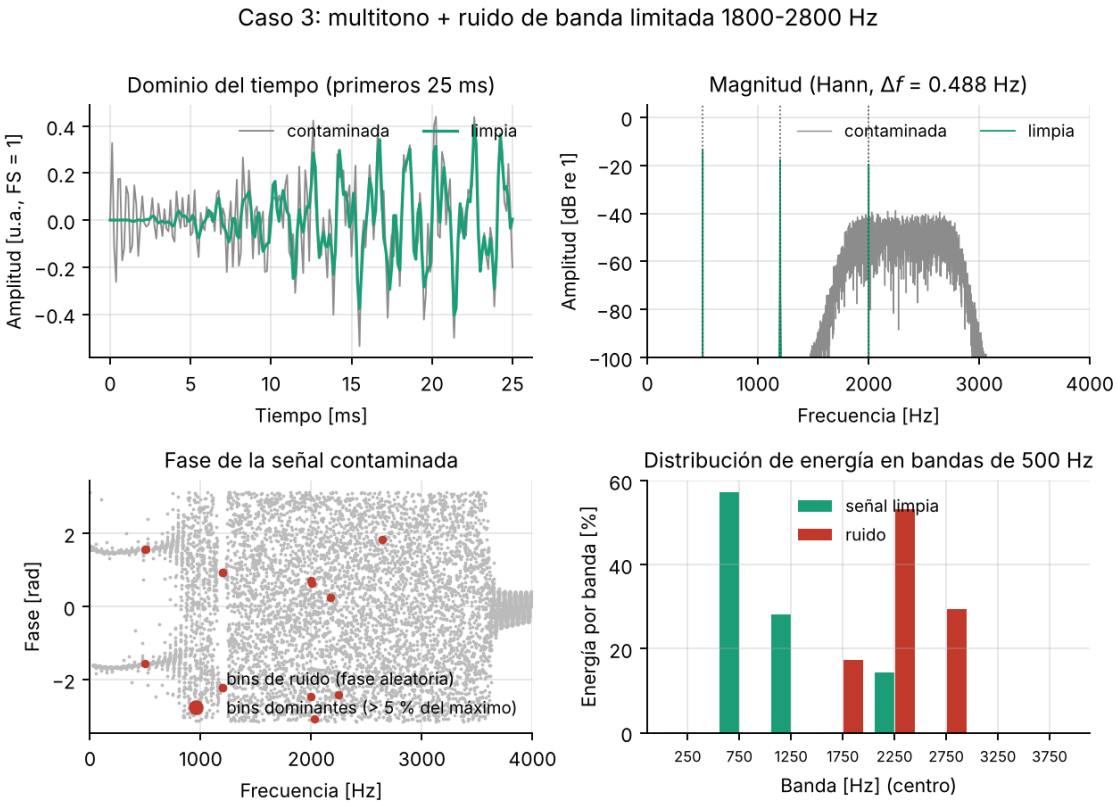


Figura 7: Caso 3: multitono + ruido de banda limitada de 1800 a 2800 Hz.

En el caso 1, la interferencia aparece como un pico aislado en 3100 Hz. En el caso 2, el ruido

blanco se distribuye por casi toda la banda. En el caso 3, el ruido ocupa de 1800 a 2800 Hz y se superpone con el tono de 2000 Hz. Esta superposición dificulta separar el ruido de la señal útil.

(d) Características para identificar el ruido automáticamente. Se calculan con el periodograma promedio (Welch [4], Hann, 50 % de solapamiento):

Caso	Planitud espectral	Ancho 99 % [Hz]	Centroide [Hz]	Pico/mediana [dB]	Ocupación [Hz]	Tipo detectado (solo con la contaminada)
1	5.4×10^{-10}	47	3100	120.1	31	interferencia tonal
2	0.993	3953	1993	1.2	1703	ruido de banda ancha
3	6.4×10^{-7}	1031	2298	91.6	516	ruido de banda limitada

El ancho de banda y la ocupación espectral permiten distinguir los tres casos. El clasificador usa solo la señal contaminada y revisa la región de 2300 a 4000 Hz: si el pico es estrecho, identifica interferencia tonal; si ocupa casi toda la región, identifica ruido de banda ancha; en los demás casos, lo clasifica como ruido de banda limitada. Los umbrales se ajustaron con estos tres ejemplos, por lo que deben probarse con otros tipos de ruido.

Información que no se recupera cuando comparten frecuencias. Cuando la señal útil y el ruido ocupan la misma frecuencia, el espectro observado es la suma de ambos. El filtro no puede distinguirlos sin información adicional. Por eso, en los casos 2 y 3 queda ruido en la banda útil; en el caso 1, como la interferencia está separada, se puede eliminar casi por completo.

3 Diseño de un filtro temporal FIR (2.3)

Se escogió el **caso 1** (interferencia en 3100 Hz, fuera de la banda útil, que llega a 2000 Hz): corresponde un filtro **pasa bajas**.

(a) Especificación. Banda de paso 0–2200 Hz (margen de 200 Hz sobre el tono más alto); banda de rechazo ≥ 2800 Hz (la interferencia queda 300 Hz dentro); transición de 600 Hz, con corte a la mitad ($f_c = 2500$ Hz); rizado de paso ≤ 0.02 dB y atenuación en rechazo ≥ 60 dB.

(b) Justificación del tipo y del orden. Se eligió un filtro FIR por tres razones: *fase lineal* (el retardo es constante, 26 muestras $= 3.25$ ms, y la forma de onda multitono no se distorsiona); *estabilidad incondicional*; y *tolerancia a la cuantización* de coeficientes. La desventaja es que necesita más coeficientes que un filtro IIR para cumplir con las mismas condiciones. Se comparó con filtros IIR, que pueden necesitar menos operaciones, pero tienen fase no lineal y su estabilidad puede verse afectada por la cuantización. Como el costo del FIR es aceptable para el sistema propuesto, se priorizó conservar la forma de onda. El diseño usa una ventana Kaiser: se añadieron 2 dB de margen porque con 60 dB teóricos no se alcanzó la atenuación requerida. El resultado final tiene 53 coeficientes (orden 52) y $\beta = 5.874$.

(c) Coeficientes y ecuación de diferencias. Por simetría $h[k] = h[M-k]$; se listan los $k = 0, \ldots, 26$:

k	$h[k] = h[52-k]$	k	$h[k] = h[52-k]$
0	+0.00014447	14	-0.01456259
1	-0.00034640	15	+0.00672029
2	+0.00000000	16	+0.01494427
3	+0.00084530	17	-0.02350463
4	-0.00092667	18	+0.00000000
5	-0.00069245	19	+0.03449841
6	+0.00242847	20	-0.03247891
7	-0.00121920	21	-0.02205083
8	-0.00290077	22	+0.07467376
9	+0.00480489	23	-0.03918055
10	-0.00000000	24	-0.11077261
11	-0.00744187	25	+0.29292344
12	+0.00698601	26	+0.62500983
13	+0.00460324		

$$y[n] = \sum_{k=0}^{52} h[k]\, x[n-k]. \tag{3}$$

(d) Función de transferencia.

$$H(z) = \sum_{k=0}^{52} h[k] z^{-k} = z^{-52} \sum_{k=0}^{52} h[k] z^{52-k}, \qquad H(e^{j\omega}) = e^{-j26\omega} \Big[h[26] + 2 \sum_{m=1}^{26} h[26-m] \cos(m\omega) \Big]. \tag{4}$$

La expresión entre corchetes es real, por lo que la fase es -26ω (lineal). Además $H(1) = \sum h[k] = 1.0000$ (ganancia unitaria en continua).

(e) Polos, ceros, ROC y estabilidad.

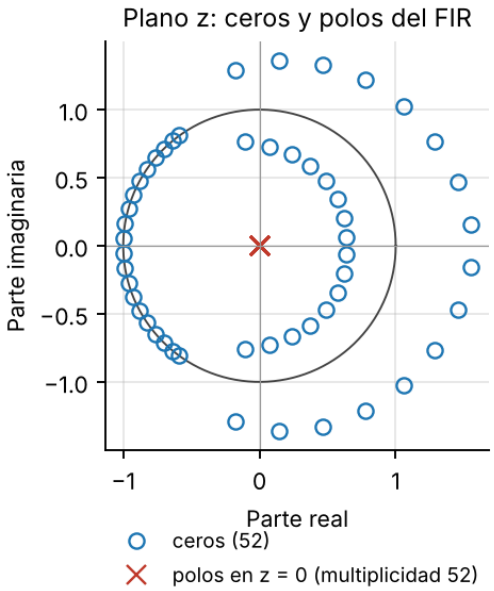


Figura 8: Plano z: 52 ceros y 52 polos en el origen.

El filtro tiene 52 ceros y 52 polos en el origen. Al ser causal, su región de convergencia es $|z| > 0$, que incluye la circunferencia unitaria. Como es FIR y sus coeficientes son finitos, es estable BIBO.

(f) Respuesta en magnitud, fase y retardo de grupo.

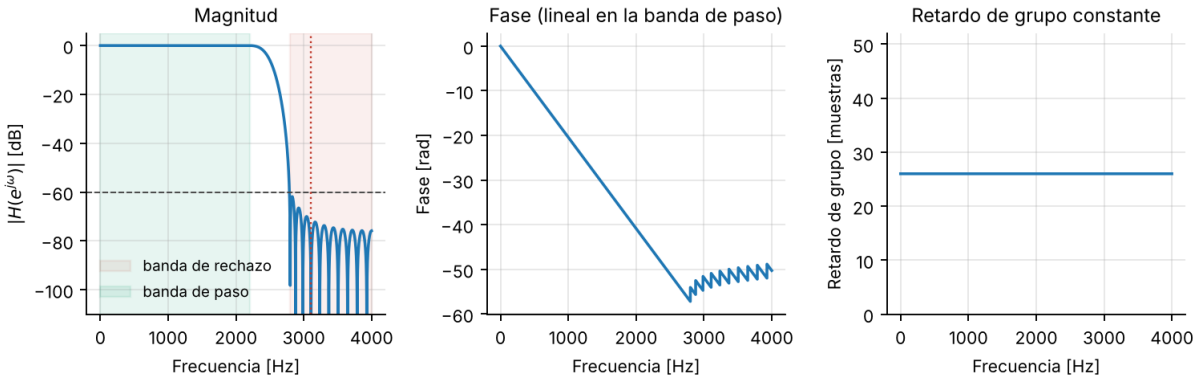


Figura 9: Respuesta del FIR: magnitud, fase y retardo de grupo (línea punteada: interferencia en 3100 Hz).

Se miden un rizado de paso de 0.0080 dB y una atenuación mínima en rechazo de 61.8 dB (cumple), con $|H(3100\text{ Hz})| = -86.7$ dB. El retardo de grupo medio en la banda de paso es 26.000 muestras, igual al teórico.

Aplicación y validación. La ecuación de diferencias se implementó con ciclos de código y se comparó con `scipy.signal.lfilter` [**?**]. En los tres casos, el error máximo fue menor que 10^{-12}, por lo que ambas implementaciones coinciden dentro de la tolerancia.

(g) Aplicación a los tres casos. Los resultados de los tres casos (diseño optimizado para el 1) están en la Sección 5 (Tabla 2 y Figs. 14 y 15). En el caso 2 el filtro solo elimina el ruido de $f > 2500$ Hz; como el ruido es blanco, conserva $2500/4000$ de su potencia y la mejora esperada es $10\log_{10}(4000/2500) = 2.04$ dB, muy cerca de los 2.2 dB medidos. En el caso 3 el ruido de 1800–2500 Hz pasa casi completo, y de ahí los 1.9 dB.

(h) Cuantización, saturación y memoria para el microcontrolador. Se simularon coeficientes de 16, 12, 10 y 8 bits (formato Q1.$(b-1)$):

Bits de coef.	$\max\lvert\Delta h\rvert$	Atenuación mín. en rechazo [dB]	$\lvert H(3100\,\text{Hz})\rvert$ [dB]	SNR de salida, caso 1 [dB]
64 (float)	0	61.8	-86.7	84.3
16	1.5×10^{-5}	62.1	-79.8	83.6
12	2.4×10^{-4}	50.7	-70.5	60.4
10	9.5×10^{-4}	37.1	-69.0	50.2
8	3.9×10^{-3}	30.3	-75.7	36.4

Con 16 bits (Q15), el filtro sigue cumpliendo con lo pedido (62.1 dB ≥ 60); con 12 bits la atenuación baja a 50.7 dB y la SNR del caso 1 de 84.3 a 60.4 dB. Por tanto, el FIR en punto fijo requiere coeficientes de al menos 16 bits. **Saturación:** $\sum|h[k]| = 2.02 > 1$, así que en el peor caso una entrada de 0.8 FS produce 1.62 FS y se saturaría. Las tres señales de prueba no saturan (picos de salida ≤ 0.70 FS), pero para garantizarlo se debe escalar la entrada a < 0.49 FS o aceptar saturación. **Memoria** (float32): coeficientes 212 B + estado 1232 B + buffers 2048 B ≈ 3.5 kB, despreciable frente a los 192 kB de SRAM. Observación: con un ADC de 12 bits la SNR máxima por cuantización es ≈ 74 dB, de modo que los 84.3 dB del FIR solo son alcanzables en simulación.

4 Reducción espectral mediante FFT e IFFT (2.4)

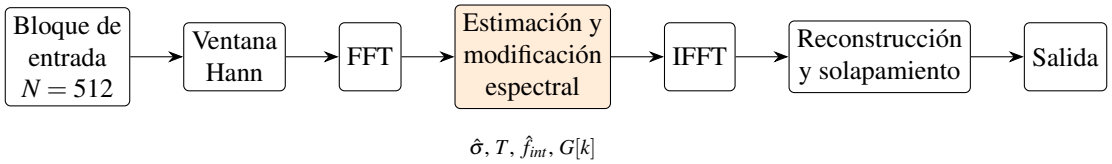


Figura 10: Cadena de procesamiento por bloques del método espectral.

Para el bloque b ($H = N/2$ es el salto, w la ventana Hann):

$$X_b[k] = \mathrm{FFT}\{w[n]\,x[bH+n]\}, \qquad \hat{\sigma}_b = \mathrm{mediana}_k\,|X_b[k]|, \qquad T_b = \max\left(\alpha\hat{\sigma}_b,\ \tau \max_k |X_b[k]|\right), \tag{5}$$

$$G_b[k] = \begin{cases} 1, & \text{si } |X_b[k]| \geq T_b \text{ o está a } \pm\text{ext bins de uno que cumple} \\ 0, & \text{en otro caso} \end{cases} \qquad G_b[N-k] = G_b[k], \tag{6}$$

$$y_b = \mathrm{IFFT}\{G_b[k]X_b[k]\}, \qquad y[n] = \sum_b y_b[n-bH]. \tag{7}$$

Además, si el pico más fuerte fuera de la banda útil (300–2300 Hz) supera en 6 veces el promedio de sus vecinos (es decir, es un tono angosto y no un pico de ruido), se toma su frecuencia como $\hat{f}_{int}$ y se anulan ± 3 bins a su alrededor. Se conservan ± 2 bins junto a cada pico (el lóbulo principal de Hann) para no recortar los tonos.

Se usaron bloques de $N = 512$, FFT radix-2 propia y ventana Hann con 50 % de solapamiento. Los parámetros se ajustan en cada bloque ($\alpha = 4$, $\tau = 0.3$ y extensión de 2 bins). La ganancia mantiene la simetría conjugada y la reconstrucción con ganancia unitaria reproduce la señal original. El método se probó en los tres casos.

Tabla 1: Comportamiento adaptativo por caso (63 bloques). La señal es de amplitud ≤ 0.8 y las magnitudes $|X|$ son sin normalizar.

Caso	Bloques con tono detectado	$\hat{f}_{int}$ [Hz]	$\hat{\sigma}$ mediana	T mediana	Bins conservados [%]	$\max\lvert\mathrm{Im}\{\mathrm{IFFT}\}\rvert$
1	63/63	3093.75 ± 0.00	0.000234	11.6	7.0	1.7×10^{-16}
2	0/63	–	1.42	8.78	7.2	1.2×10^{-16}
3	0/63	–	0.00276	7.99	7.4	1.3×10^{-16}

En el caso 1, el tono se detectó en los 63 bloques con una estimación cercana a 3100 Hz. En los casos 2 y 3 no hubo falsas detecciones, porque el ruido no cumplió el criterio de prominencia de un tono aislado.

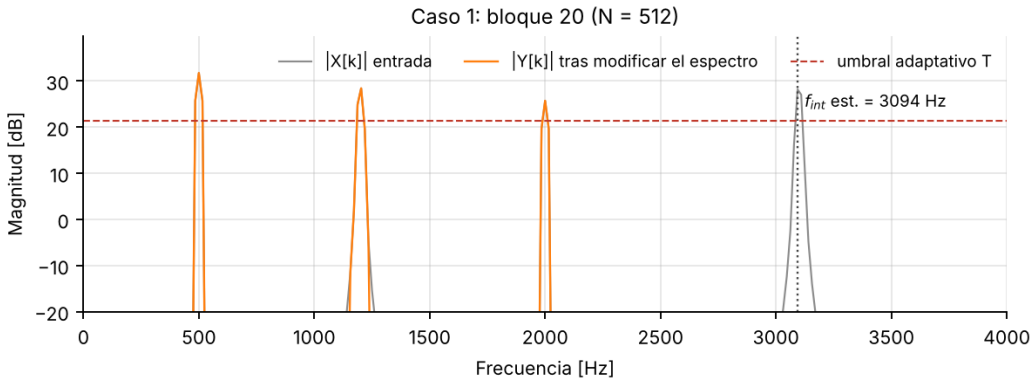


Figura 11: Un bloque del caso 1: entrada, umbral adaptativo T, interferencia estimada y salida.

f) Caso en que eliminar más ruido degrada la señal útil. Se barrieron el umbral sobre el nivel de ruido α (caso 2) y el umbral relativo al pico τ (caso 3):

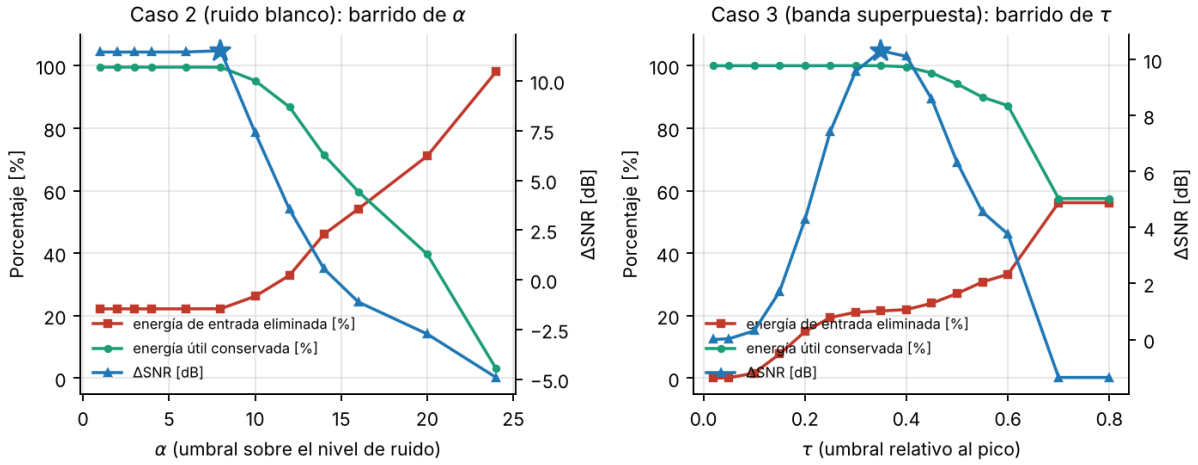


Figura 12: Compromiso entre eliminar energía y conservar la señal útil. La estrella marca el máximo de ΔSNR.

Barrido	Valor	Energía de entrada eliminada [%]	Energía útil conservada [%]	ΔSNR [dB]
α (caso 2)	4	22.2	99.5	+11.4
α (caso 2)	8	22.2	99.5	+11.5
α (caso 2)	12	32.9	86.8	+3.5
α (caso 2)	16	54.2	59.7	-1.1
α (caso 2)	24	98.1	3.1	-4.9
τ (caso 3)	0.3	21.1	100.0	+9.6
τ (caso 3)	0.4	22.0	99.6	+10.1
τ (caso 3)	0.5	27.1	94.2	+6.3
τ (caso 3)	0.6	33.2	87.2	+3.8
τ (caso 3)	0.8	56.2	57.5	-1.4

Los resultados muestran que un umbral moderado elimina ruido sin afectar mucho los tonos útiles. Para el caso 2, el mejor resultado del barrido fue una mejora de 11.5 dB con $\alpha = 8$. Si el umbral aumenta demasiado, también elimina tonos: con $\alpha = 24$ solo se conserva el 3.1 % de la energía útil. En el caso 3, un umbral alto también reduce la señal deseada. Por tanto, eliminar más energía no siempre significa obtener un mejor resultado.

5 Comparación y propuesta para el sistema embebido (2.5)

Tabla 2: Métricas de los tres casos (segmento interior). La atenuación se mide en la banda de la interferencia dominante: caso 1, 3050–3150 Hz; caso 2, 2300–4000 Hz (solo ruido); caso 3, 1800–2800 Hz.

Caso	Método	MSE	SNR ent. [dB]	SNR sal. [dB]	ΔSNR [dB]	E. útil [%]	Atenuación interf. [dB]
1	Entrada	2.5×10^{-2}	5.05	5.05	0.00	100.0	0.0
1	FIR	2.9×10^{-10}	5.05	84.28	79.23	100.0	86.7
1	Espectral	3.3×10^{-7}	5.05	53.77	48.72	100.0	94.9
2	Entrada	1.4×10^{-2}	5.07	5.07	0.00	99.5	0.0
2	FIR	8.7×10^{-3}	5.07	7.22	2.15	99.5	11.1
2	Espectral	1.0×10^{-3}	5.07	16.51	11.45	99.5	46.6
3	Entrada	1.2×10^{-2}	5.07	5.07	0.00	100.0	-0.0
3	FIR	7.7×10^{-3}	5.07	6.92	1.85	100.0	1.9
3	Espectral	1.3×10^{-3}	5.07	14.65	9.58	100.0	9.3

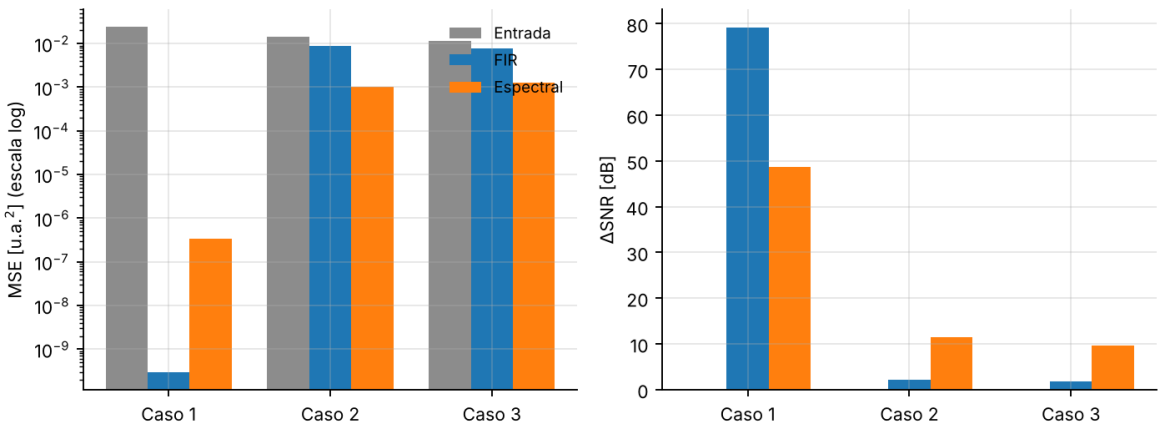


Figura 13: MSE y ΔSNR por caso y método.

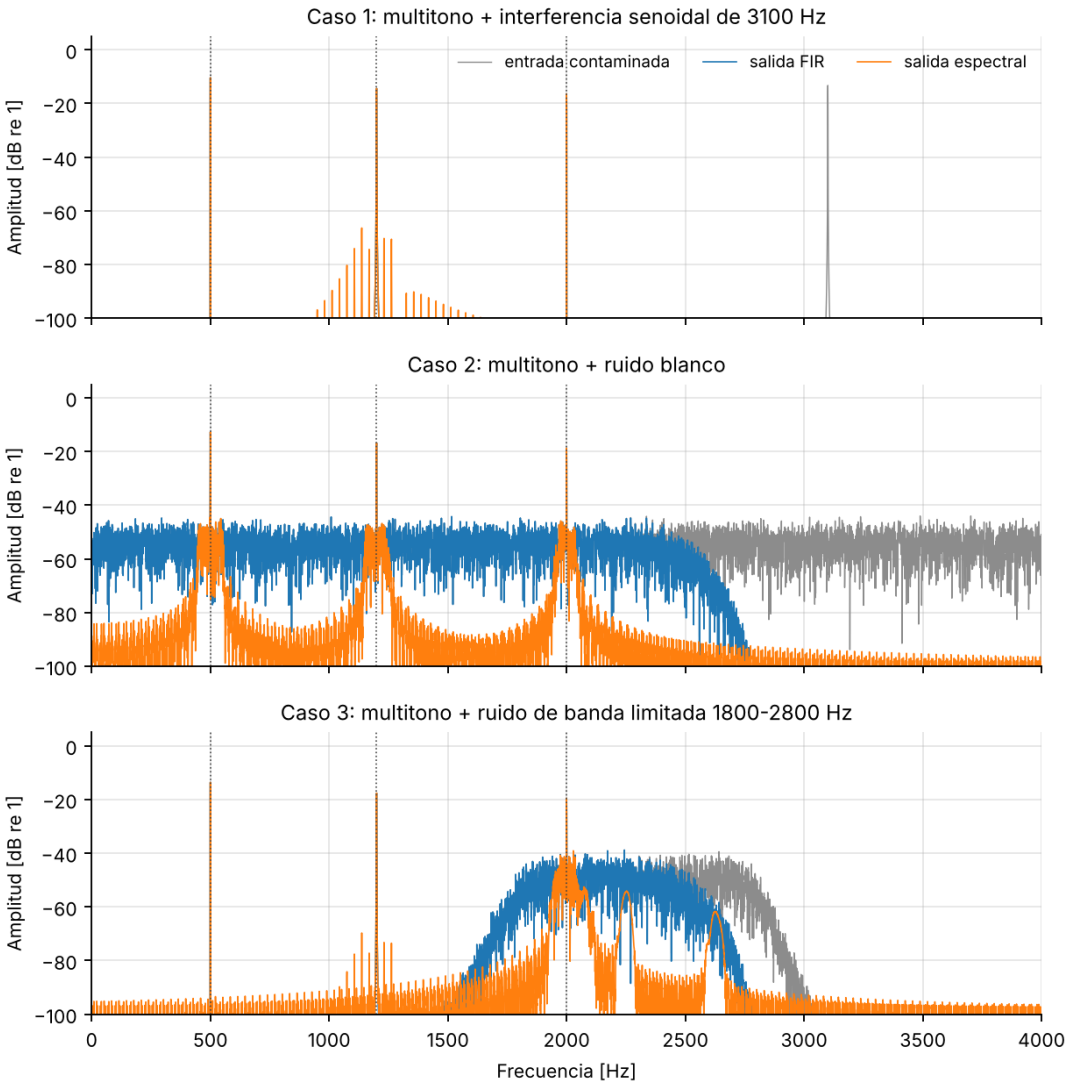


Figura 14: Espectro de la entrada y de las dos salidas (Hann, $\Delta f = 0.488$ Hz).

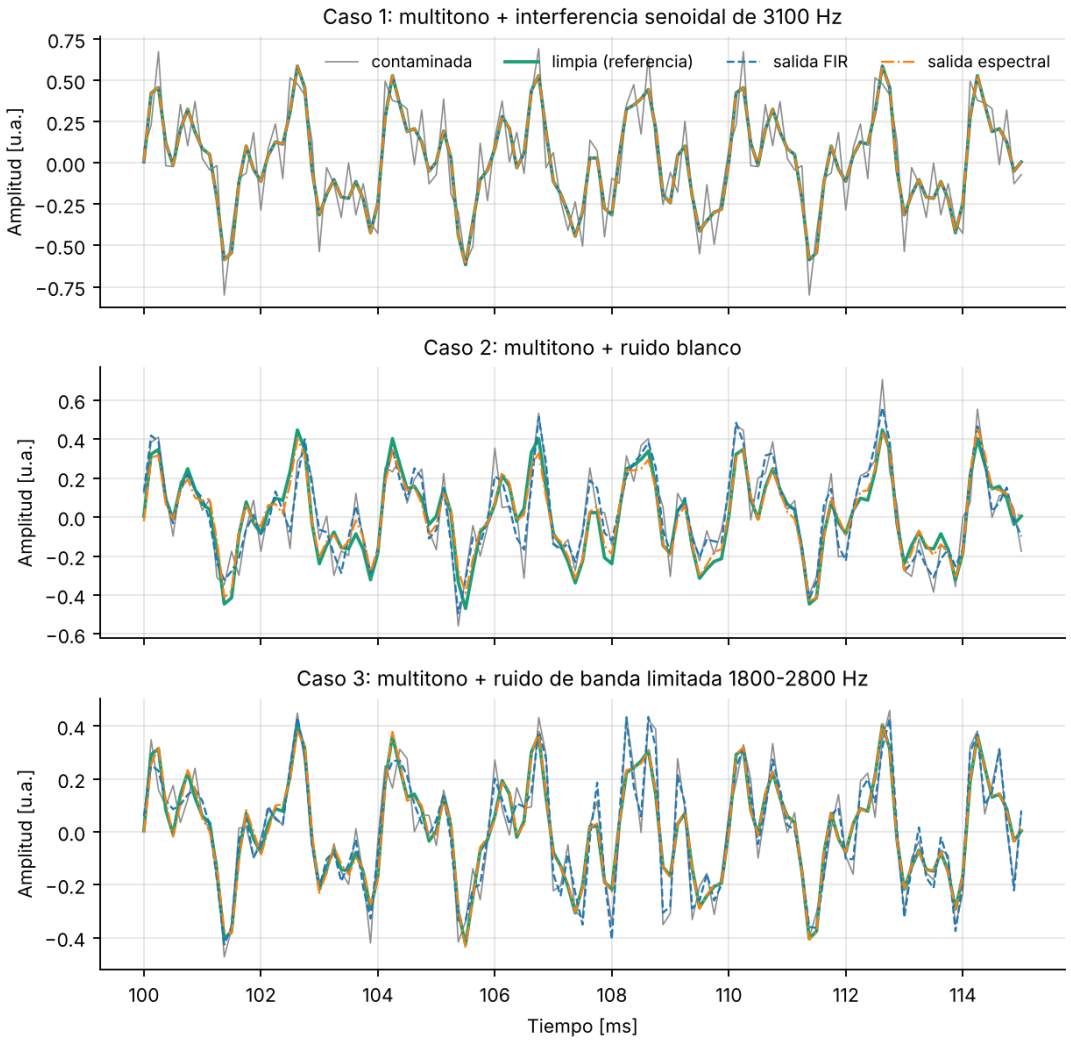


Figura 15: Forma de onda (15 ms): referencia limpia, contaminada y salidas (FIR con retardo compensado).

Nota sobre la energía útil. Es ≈ 100 %; las pequeñas diferencias (por ejemplo 99.5 % en el caso 2) aparecen porque el ruido y la señal no son perfectamente ortogonales en una sola realización (la correlación cruzada aleatoria es de ± 1 %).

Tabla 3: Costo por salto de 256 muestras (32 ms). PC: Python (referencia relativa). MCU: estimación con 3 ciclos por operación a 168 MHz (supuesto).

Método	Operaciones por salto	PC [ms] (propia / biblioteca)	MCU estimado [ms]	RAM aprox. [B]
FIR (53 coef.)	13 568	2.76 / 0.021	0.24	3 492
Espectral ($N = 512$)	32 296	0.44 / 0.112	0.58	8 196

a) ¿Qué método ofrece el mejor compromiso? Depende del ruido. En el caso 1, el FIR logra la mayor mejora porque la interferencia está fuera de la banda útil. En los casos 2 y 3, el método espectral mejora más la SNR porque conserva los tonos y elimina parte del ruido distribuido en frecuencia. El FIR requiere menos memoria y tiene menor latencia; el método espectral es más flexible, pero procesa bloques más largos. Por eso se propone usar el método espectral como opción general y el FIR cuando se detecte una interferencia tonal fuera de banda.

b) ¿En qué casos falla cada método y por qué? El FIR no elimina bien el ruido que cae dentro de su banda de paso y no se adapta a cambios de frecuencia. Además, con coeficientes de menos de 16 bits puede incumplir la atenuación y en punto fijo puede saturarse. El método espectral puede eliminar tonos útiles si el umbral es demasiado alto; también tiene dificultades con señales cambiantes y cuando el ruido comparte frecuencias con la señal. El recorte espectral puede producir artefactos que deben evaluarse en el hardware [6].

c) Tamaño de bloque, frecuencia de muestreo y formato numérico propuestos. $f_s = 8$ kHz (justificada en la introducción), bloque $N = 512$ con salto 256: $\Delta f = 15.6$ Hz resuelve los tonos de la prueba, la latencia de bloque es de 64 ms (salto de 32 ms) y la RAM es de 8.2 kB. $N = 1024$ resolvería 30 Hz pero duplica latencia y memoria; $N = 256$ ($\Delta f = 31$ Hz) mezcla los tonos cercanos. Formato: *transporte en int16 (Q15)*; ADC y DAC de 12 bits (los 12 bits dan un piso de $\approx$74 dB, muy por encima de las SNR de entrada de 5 dB); *float32* en la FFT y el FIR del STM32F407, que tiene FPU; si se usara punto fijo, coeficientes de 16 bits (Q15), acumulador de 32 o 64 bits y entrada a < 0.49 FS. La carga estimada es de 0.8 % (FIR) y 1.8 % (espectral) del salto de 32 ms, con mucho margen.

d) Bibliotecas disponibles para el hardware escogido. Se propone usar CMSIS-DSP [**?**], ya que está optimizada para microcontroladores Arm y ofrece funciones para FFT y filtros FIR/IIR. KISS FFT [**?**] es otra opción, aunque no incluye los filtros y puede ser menos rápida en este hardware. La implementación final debe comprobarse en el microcontrolador real.

e) Diagrama de bloques definitivo de los tres nodos.

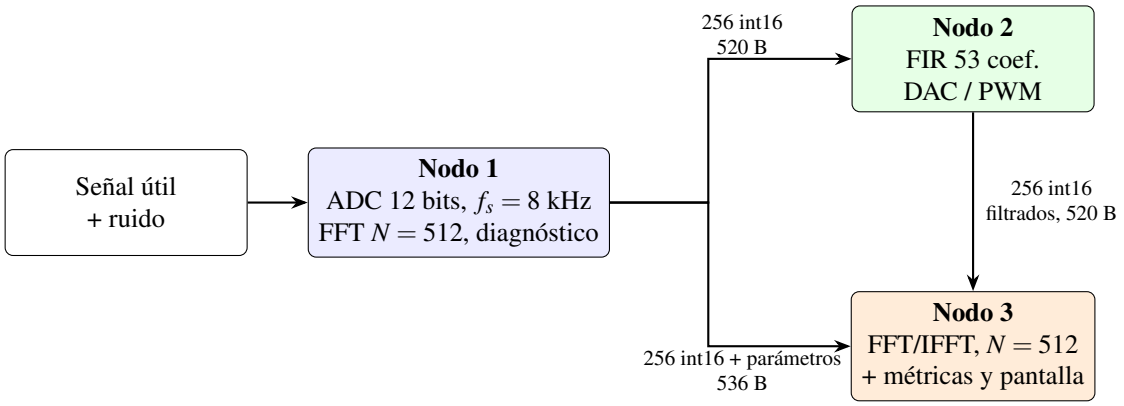


Figura 16: Arquitectura de tres nodos. Los bytes son por salto de 256 muestras (32 ms).

La comunicación puede usar UART 8N1 con sincronización, número de secuencia y verificación CRC para detectar errores y bloques perdidos. El enlace más exigente necesita al menos 167.5 kbit/s; por eso se propone UART a 460800 baudios. Una trama de 520 B tarda cerca de 11.3 ms, por debajo del salto de 32 ms. La latencia total depende del tamaño de bloque y del método utilizado.

6 Uso de IA

Se utilizó Claude y Codex en Visual Studio Code para apoyar el desarrollo del código, especialmente en la generación de las figuras, se suministró el diseño y analisis realizado previamente al desarrollo del código y los resultados esperados. Para la validación se generaron pruebas, los resultados se comprobaron con 36 pruebas automáticas, comparaciones con `numpy` y `scipy` y se verificaron contra la teoría. También se utilizó como apoyo en la generación del pdf.

Referencias

[1] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[2] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.

[3] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978.

[4] P. D. Welch, "The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms," *IEEE Trans. Audio Electroacoust.*, vol. AU-15, no. 2, pp. 70–73, Jun. 1967.

[5] J. B. Allen and L. R. Rabiner, "A unified approach to short-time Fourier analysis and synthesis," *Proc. IEEE*, vol. 65, no. 11, pp. 1558–1564, Nov. 1977.

[6] S. F. Boll, "Suppression of acoustic noise in speech using spectral subtraction," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. ASSP-27, no. 2, pp. 113–120, Apr. 1979.

[7] L. Chavarría Zamora, "CE1110: Generador de señales," *Kaggle Notebook*, `https://www.kaggle.com/code/luischavarriazamora/ce1110-generador-de-se-ales`.